ILA (Iksha Learning Academy)

Rails and React Training Curriculum for a one-year experience engineer. Each assignment page includes topic context, visual learning map, reading material, expected outcomes, and self-check questions.

Completion Window: 2-3 weeks | Assignments: 18 | Track: Rails-focused with React implementation | Certification: Level 1, Mentor-reviewed

How to Navigate

1. Open assignment page from the table of contents.
2. Complete prerequisite reading before implementation.
3. Execute assignment tasks and record evidence in `docs/assignment-XX.md`.
4. Use completion criteria and self-check questions before submitting for review.

Completion and Certification Policy

- The curriculum is designed for completion within **2 to 3 weeks**.
- All 18 assignments must be submitted with evidence as defined on each assignment page.
- A final review is conducted by an assigned mentor after all submissions are complete.
- Certification is awarded only after the learner **passes the mentor final review**.
- Certification title: **ILA Rails and React Engineering Certification - Level 1**

Learning Journey

Phase 1
Foundation setup, CLI workflow, Git discipline

Phase 2
Rails domain, API design, testing, background jobs

Phase 3
React UX delivery, frontend testing, full-stack feature

Phase 4
Performance, CI gates, operations, capstone defense

Table of Contents

ID	Assignment	Page
01	Rails and React Local Setup	Open
02	Command-Line and Rails CLI Proficiency	Open
03	Git Workflow and Recovery	Open
04	Rails Controller Architecture and Service Layer	Open
05	Data Model, Validations, and Migrations	Open
06	Rails API Endpoints and HTTP Semantics	Open
07	Backend Test Suite (Models, Services, Controllers)	Open
08	Background Jobs with Sidekiq	Open
09	Error Handling and Request Observability	Open
10	React Application Shell and Routing	Open
11	Appointments List UI and State Handling	Open
12	Appointment Create/Edit Form	Open
13	Frontend Testing (Jest and RTL)	Open
14	End-to-End Feature Delivery (Rails and React)	Open
15	Database and API Performance Optimization	Open
16	CI Pipeline and Quality Gates	Open
17	Production Readiness and Operational Runbook	Open
18	Capstone Demonstration and Technical Defense	Open

Assignment Overview Cards

Assignment 01: Rails and React Local Setup

This assignment establishes your development base. You will configure Rails and React projects, verify application startup, and confirm frontend to backend connectivity.

Assignment 02: Command-Line and Rails CLI Proficiency

This assignment improves command-line fluency for daily engineering operations including service checks, cleanup, and development diagnostics.

Assignment 03: Git Workflow and Recovery

This assignment focuses on branch discipline, conflict resolution, and commit recovery so code history remains clean and reliable.

Assignment 04: Rails Controller Architecture and Service Layer

This assignment introduces service objects to keep controllers focused on HTTP concerns and move business rules to dedicated domain units.

Assignment 05: Data Model, Validations, and Migrations

This assignment designs the domain schema and validation boundaries for patient, provider, and appointment data.

Assignment 06: Rails API Endpoints and HTTP Semantics

This assignment delivers RESTful endpoints for appointment lifecycle operations with proper status codes and response consistency.

Assignment 07: Backend Test Suite (Models, Services, Controllers)

This assignment adds deterministic backend tests that cover model rules, service decisions, and controller responses.

Assignment 08: Background Jobs with Sidekiq

This assignment introduces asynchronous job processing for non-blocking notification workflows using Sidekiq and Redis.

Assignment 09: Error Handling and Request Observability

This assignment standardizes API error handling and introduces request correlation for easier debugging.

Assignment 10: React Application Shell and Routing

This assignment builds the frontend shell, route structure, and reusable API client foundation.

Assignment 11: Appointments List UI and State Handling

This assignment implements data listing with filters and clear UI states for loading empty and failure conditions.

Assignment 12: Appointment Create/Edit Form

This assignment builds reusable form workflows for create and edit operations with validation feedback.

Assignment 13: Frontend Testing (Jest and RTL)

This assignment introduces frontend test coverage for render states, user behavior, and failure conditions.

Assignment 14: End-to-End Feature Delivery (Rails and React)

This assignment delivers one complete feature across backend and frontend: appointment reschedule with conflict protection.

Assignment 15: Database and API Performance Optimization

This assignment applies measurement-based optimization to improve one backend endpoint performance path.

Assignment 16: CI Pipeline and Quality Gates

This assignment establishes automated quality checks for tests and linting in a repeatable CI workflow.

Assignment 17: Production Readiness and Operational Runbook

This assignment documents deployment safeguards and incident handling steps for common operational failures.

Assignment 18: Capstone Demonstration and Technical Defense

This assignment synthesizes all prior work into a complete demonstration with architecture rationale and evidence-based defense.

Evaluation Rubric (Per Assignment)

Category	Score	Expectation
Correctness	/5	Required behavior is implemented accurately including edge cases.
Code Quality	/5	Code is readable maintainable and appropriately structured.
Testing	/5	Success and failure paths are covered with deterministic tests.
Debuggability	/5	Logs and errors enable quick issue diagnosis.
Judgment	/5	Tradeoffs and scope decisions are practical and defensible.

Assignment 01: Rails and React Local Setup

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track Guided Assignment Page

Topic Overview

This assignment establishes your development base. You will configure Rails and React projects, verify application startup, and confirm frontend to backend connectivity.

Why This Matters in Real Projects

Unstable local environments waste team time and create false bugs. A disciplined setup ensures every future task starts from a predictable baseline.

Visual Learning Map

Step 1
Initialize backend and frontend

Step 2
Add health endpoint and API connectivity

Step 3
Script startup and shutdown flow

Step 4
Capture reproducible setup evidence

Prerequisite Reading (45-60 min)

- Rails Guides: Getting Started
- React Docs: Learn React
- Docker Docs: Compose Overview

Assignment Tasks

1. Create Rails app in backend and React app in frontend
2. Create backend health endpoint and display status on UI
3. Add scripts/start-all.sh and scripts/stop-all.sh

Expected Deliverables

- README with exact commands
- docs/assignment-01.md evidence log
- Screenshot of healthy backend and frontend

What You Will Learn After This

- Bootstrap local full stack projects cleanly
- Verify cross-service connectivity quickly
- Package setup as repeatable documentation

Common Mistakes to Avoid

- Skipping environment version checks
- Hardcoding local paths in scripts
- Submitting without reproducible startup proof

Avoid these issues before requesting review.

Completion Criteria

- Fresh clone runs from README commands
- Frontend confirms live backend health state
- All required files exist in expected folders

Review should be possible without extra clarifications.

Self-Check Questions

1. Can another engineer run this in under 15 minutes
2. What fails first if backend is down
3. Where is setup evidence documented

← Back to Table of Contents | Table of Contents | Next Assignment →

Assignment 02: Command-Line and Rails CLI Proficiency

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track Guided Assignment Page

Topic Overview

This assignment improves command-line fluency for daily engineering operations including service checks, cleanup, and development diagnostics.

Why This Matters in Real Projects

Most debugging and deployment workflows depend on CLI literacy. Faster terminal operations improve delivery speed and reduce context switching.

Visual Learning Map

Step 1
Build status script for services

Step 2
Build safe cleanup utility

Step 3
Document essential commands with purpose

Step 4
Validate scripts on current environment

Prerequisite Reading (40-50 min)

- Rails Guides: The Rails Command Line
- MIT Missing Semester: Shell Tools
- GNU Bash Manual

Assignment Tasks

1. Create scripts/dev-status.sh for app and DB checks
2. Create scripts/clean-safe.sh for temporary artifact cleanup
3. Document 25 commands used in this project

Expected Deliverables

- docs/assignment-02.md command catalogue
- Script outputs for both scripts
- Short explanation of command categories

What You Will Learn After This

- Automate repeated local checks
- Use Rails CLI confidently for project tasks
- Differentiate safe cleanup from destructive cleanup

Common Mistakes to Avoid

- Using destructive cleanup commands
- Not handling missing processes in scripts
- Listing commands without context

Avoid these issues before requesting review.

Completion Criteria

- Scripts run without manual edits
- Command documentation is accurate and practical
- Safety assumptions are clearly stated

Review should be possible without extra clarifications.

Self-Check Questions

1. How would you diagnose backend not booting
2. Which command verifies routes quickly
3. What cleanup operation is intentionally excluded

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 03: Git Workflow and Recovery

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track Guided Assignment Page

Topic Overview

This assignment focuses on branch discipline, conflict resolution, and commit recovery so code history remains clean and reliable.

Why This Matters in Real Projects

Poor Git practices increase review friction and risk of code loss. Teams rely on clean history for debugging, rollback, and auditability.

Visual Learning Map

Step 1
Create branch divergence scenario

Step 2
Resolve conflict with rebase

Step 3
Recover dropped commit using reflog

Step 4
Document final history

Prerequisite Reading (35-45 min)

- Pro Git Book
- Git Docs: git-rebase
- Git Docs: git-reflog

Assignment Tasks

1. Create two branches with intentional merge conflict
2. Resolve conflict with rebase while preserving intended code
3. Recover one dropped commit using reflog

Expected Deliverables

- docs/assignment-03.md with command transcript
- Commit graph screenshot
- Brief note on branch strategy

What You Will Learn After This

- Rebase safely in collaborative workflows
- Recover work confidently after history edits
- Prepare review-friendly commit sequences

Common Mistakes to Avoid

- Force pushing without understanding consequences
- Resolving conflict without testing behavior
- Submitting tangled commit history

Avoid these issues before requesting review.

Completion Criteria

- Final history is linear and readable
- Recovered commit is demonstrably restored
- Conflict resolution preserves intended behavior

Review should be possible without extra clarifications.

Self-Check Questions

1. What signals that rebase was successful
2. How would you recover from accidental reset
3. When should merge be preferred over rebase

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 04: Rails Controller Architecture and Service Layer

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track Guided Assignment Page

Topic Overview

This assignment introduces service objects to keep controllers focused on HTTP concerns and move business rules to dedicated domain units.

Why This Matters in Real Projects

Thin controllers and explicit service layers improve maintainability, testability, and onboarding speed for other engineers.

Visual Learning Map

Step 1
Define service responsibility

Step 2
Move business logic out of controller

Step 3
Return consistent success and failure result

Step 4
Test domain rules separately

Prerequisite Reading (45-60 min)

- Rails Guides: Action Controller Overview
- Rails Guides: Routing
- Ruby Docs: Modules and Classes

Assignment Tasks

1. Create Appointments::Create service in app/services
2. Update controller to delegate booking behavior
3. Implement deterministic overlap conflict checks

Expected Deliverables

- backend/app/services/appointments/create.rb
- Controller refactor evidence
- docs/assignment-04.md with architecture notes

What You Will Learn After This

- Separate HTTP and business layers cleanly
- Design service interfaces that are easy to test
- Enforce domain invariants in one location

Common Mistakes to Avoid

- Leaving business logic in controller actions
- Returning inconsistent result structures
- Coupling service code tightly to HTTP objects

Avoid these issues before requesting review.

Completion Criteria

- Controller remains thin and readable
- Service handles business rules and conflicts
- Service behavior is testable in isolation

Review should be possible without extra clarifications.

Self-Check Questions

1. What belongs in controller versus service
2. How does this improve test speed
3. How does service return failure details

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 05: Data Model, Validations, and Migrations

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track Guided Assignment Page

Topic Overview

This assignment designs the domain schema and validation boundaries for patient, provider, and appointment data.

Why This Matters in Real Projects

Correct data modeling prevents silent corruption and reduces downstream bug cost. Teams depend on predictable persistence behavior.

Visual Learning Map

Step 1
Define core entities

Step 2
Add model-level validation rules

Step 3
Add database-level constraints and indexes

Step 4
Validate migration rollback flow

Prerequisite Reading (50-70 min)

- Rails Guides: Active Record Basics
- Rails Guides: Active Record Validations
- Rails Guides: Active Record Migrations

Assignment Tasks

1. Create Patient Provider Appointment models
2. Add presence uniqueness and time validations
3. Add migrations with indexes and constraints

Expected Deliverables

- Model files and migration files
- docs/assignment-05.md with schema rationale
- Command outputs for migrate and rollback

What You Will Learn After This

- Design stable relational models
- Apply model and database validation boundaries
- Use migrations safely during iterative development

Common Mistakes to Avoid

- Relying only on model validations
- Skipping rollback test for migrations
- Missing indexes on query-critical fields

Avoid these issues before requesting review.

Completion Criteria

- Invalid records fail at both model and DB levels
- Migrations run up and down without manual fixes
- Schema aligns with appointment workflows

Review should be possible without extra clarifications.

Self-Check Questions

1. Which constraint must exist at DB layer
2. How do you validate migration reversibility
3. How will this schema scale for reporting

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 06: Rails API Endpoints and HTTP Semantics

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track Guided Assignment Page

Topic Overview

This assignment delivers RESTful endpoints for appointment lifecycle operations with proper status codes and response consistency.

Why This Matters in Real Projects

API correctness is foundational for frontend reliability, integration stability, and clear failure handling across systems.

Visual Learning Map

Step 1
Define route contract

Step 2
Implement CRUD and cancel operation

Step 3
Apply pagination and filtering

Step 4
Standardize error response format

Prerequisite Reading (40-55 min)

- Rails Guides: API-only Applications
- MDN: HTTP Request Methods
- MDN: HTTP Status Codes

Assignment Tasks

1. Implement index show create update cancel APIs
2. Support date range and pagination filters
3. Return uniform JSON error objects

Expected Deliverables

- Updated routes and appointments controller
- docs/assignment-06.md with sample requests
- Status code matrix

What You Will Learn After This

- Use HTTP semantics precisely
- Design APIs that frontend can consume predictably
- Document endpoint behavior for reviewers

Common Mistakes to Avoid

- Returning 200 for all responses
- Inconsistent error payload structure
- Ignoring idempotency in cancel operation

Avoid these issues before requesting review.

Completion Criteria

- Status codes align with operation outcomes
- Cancel endpoint is idempotent
- Request and response examples are documented

Review should be possible without extra clarifications.

Self-Check Questions

1. Which status should conflict return
2. How does pagination contract look
3. How is cancel request handled twice

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 07: Backend Test Suite (Models, Services, Controllers)

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track Guided Assignment Page

Topic Overview

This assignment adds deterministic backend tests that cover model rules, service decisions, and controller responses.

Why This Matters in Real Projects

Reliable tests reduce regression risk and increase confidence for refactoring and feature growth.

Visual Learning Map

Step 1
Define test coverage map

Step 2
Write model and service edge-case tests

Step 3
Write controller success and failure tests

Step 4
Run full suite and summarize

Prerequisite Reading (45-60 min)

- Rails Guides: Testing Rails Applications
- Rails Guides: Debugging Rails Applications
- Minitest Repository

Assignment Tasks

1. Add model tests for validation boundaries
2. Add service tests for overlap conflicts
3. Add controller tests for expected response codes

Expected Deliverables

- backend/test models services controllers
- docs/assignment-07.md test summary
- Evidence of green test run

What You Will Learn After This

- Design deterministic backend tests
- Translate business rules into assertions
- Detect regressions early in development

Common Mistakes to Avoid

- Using brittle time-dependent assertions
- Testing implementation details instead of behavior
- Missing failure path assertions

Avoid these issues before requesting review.

Completion Criteria

- Test suite passes consistently
- Each layer has both positive and negative coverage
- Test documentation explains run command

Review should be possible without extra clarifications.

Self-Check Questions

1. Which failure path has highest risk
2. How do you avoid flaky time tests
3. What behavior is validated in service tests

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 08: Background Jobs with Sidekiq

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track | Guided Assignment Page

Topic Overview

This assignment introduces asynchronous job processing for non-blocking notification workflows using Sidekiq and Redis.

Why This Matters in Real Projects

Production systems rely on background jobs to keep request latency low and to isolate retryable external work.

Visual Learning Map

Step 1
Configure Sidekiq and Redis

Step 2
Define notification job contract

Step 3
Trigger jobs from service layer

Step 4
Validate retry and failure behavior

Prerequisite Reading (35-50 min)

- Sidekiq Wiki: Getting Started
- Rails Guides: Active Job Basics
- Redis Docs: Getting Started

Assignment Tasks

1. Configure Sidekiq process and Redis dependency
2. Create appointment_notification_job with retry policy
3. Enqueue job on create and cancel events

Expected Deliverables

- Job class and Sidekiq configuration
- docs/assignment-08.md with queue evidence
- Sample logs for success and retry

What You Will Learn After This

- Implement asynchronous workflows responsibly
- Design retry-safe job behavior
- Observe queue execution through logs

Common Mistakes to Avoid

- Running heavy logic in controller request path
- Ignoring retry side effects
- Omitting observability for failed jobs

Avoid these issues before requesting review.

Completion Criteria

- Job execution is visible in logs
- Retries are configured and verifiable
- User-facing request remains fast

Review should be possible without extra clarifications.

Self-Check Questions

1. Why should notifications be async
2. How does retry avoid duplicate effects
3. What evidence proves job execution

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 09: Error Handling and Request Observability

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track | Guided Assignment Page

Topic Overview

This assignment standardizes API error handling and introduces request correlation for easier debugging.

Why This Matters in Real Projects

Without traceable errors, production incidents take longer to diagnose and fix. Consistent error contracts reduce confusion.

Visual Learning Map

Step 1
Add request correlation ID

Step 2
Centralize exception mapping

Step 3
Define stable error response contract

Step 4
Capture example logs

Prerequisite Reading (40-55 min)

- Rails Guides: Error Reporting
- Rails Guides: Debugging
- The Twelve-Factor App: Logs

Assignment Tasks

1. Add request ID in logs and API responses
2. Implement global exception handler in API base controller
3. Prevent internal stack traces in client responses

Expected Deliverables

- Base controller error handling code
- docs/assignment-09.md with log and response samples
- Updated API error schema

What You Will Learn After This

- Map exceptions to usable API errors
- Trace requests across logs and responses
- Improve debugging speed for failed requests

Common Mistakes to Avoid

- Returning raw exception text to clients
- Inconsistent error payload keys
- No link between log lines and API responses

Avoid these issues before requesting review.

Completion Criteria

- Every error response carries request ID
- Error schema remains consistent across endpoints
- Logs are searchable by request ID

Review should be possible without extra clarifications.

Self-Check Questions

1. How do you trace one failing request
2. Which errors should be hidden from clients
3. How is schema consistency enforced

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 10: React Application Shell and Routing

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track Guided Assignment Page

Topic Overview

This assignment builds the frontend shell, route structure, and reusable API client foundation.

Why This Matters in Real Projects

A clear route architecture reduces frontend complexity and enables faster feature additions with less rework.

Visual Learning Map

Step 1
Define route map

Step 2
Create layout and navigation scaffold

Step 3
Add API client abstraction

Step 4
Validate routing states

Prerequisite Reading (40-55 min)

- React Docs: Learn React
- React Router Tutorial
- React Docs: Sharing State

Assignment Tasks

1. Create routes for dashboard list and detail pages
2. Implement top navigation and shared layout
3. Create reusable API client module

Expected Deliverables

- Route modules and app shell code
- API client module and usage
- docs/assignment-10.md with screenshots

What You Will Learn After This

- Design scalable route architecture
- Separate navigation from page logic
- Reuse API access patterns consistently

Common Mistakes to Avoid

- Hardcoding endpoint calls in many components
- No fallback route handling
- Tight coupling between pages and layout

Avoid these issues before requesting review.

Completion Criteria

- Routes load without runtime errors
- Navigation is consistent across pages
- API client is reused not duplicated

Review should be possible without extra clarifications.

Self-Check Questions

1. How can a new page be added cleanly
2. Where should API base URL live
3. What happens on unknown route

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 11: Appointments List UI and State Handling

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track Guided Assignment Page

Topic Overview

This assignment implements data listing with filters and clear UI states for loading empty and failure conditions.

Why This Matters in Real Projects

Well-defined UI states improve user trust and reduce support burden when network or backend conditions are unstable.

Visual Learning Map

Step 1
Fetch list data and render table

Step 2
Add provider and date filters

Step 3
Handle loading empty and error states explicitly

Step 4
Validate UX behavior

Prerequisite Reading (35-50 min)

- React Docs: Conditional Rendering
- React Docs: Synchronizing with Effects
- Redux Toolkit Quick Start

Assignment Tasks

1. Implement appointments list page and filter controls
2. Render loading empty and error state components
3. Wire API data and refresh behavior

Expected Deliverables

- Frontend list page module
- State handling implementation notes
- docs/assignment-11.md with screenshots

What You Will Learn After This

- Design resilient list UIs
- Handle async state transitions cleanly
- Build predictable filtering behavior

Common Mistakes to Avoid

- Showing blank screen on failure
- Mixing filter logic with rendering logic
- Not resetting pagination or state on filter changes

Avoid these issues before requesting review.

Completion Criteria

- All three states are visible and testable
- Filters update results as expected
- User has clear recovery action on errors

Review should be possible without extra clarifications.

Self-Check Questions

1. What does user see during slow API response
2. How are empty and error states different
3. How are filters reset

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 12: Appointment Create/Edit Form

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track | Guided Assignment Page

Topic Overview

This assignment builds reusable form workflows for create and edit operations with validation feedback.

Why This Matters in Real Projects

Form quality directly affects data integrity and user productivity. Poor form handling leads to invalid records and user frustration.

Visual Learning Map

Step 1
Create reusable form component

Step 2
Apply client-side validation

Step 3
Handle server-side validation response

Step 4
Support both create and edit mode

Prerequisite Reading (35-50 min)

- React Docs: Input Component
- React Docs: Managing State
- Rails Guides: Active Record Validations

Assignment Tasks

1. Create appointment form component and shared field logic
2. Implement create page and edit page using same form
3. Map backend validation errors to inline field messages

Expected Deliverables

- AppointmentForm component and pages
- Validation handling evidence
- docs/assignment-12.md

What You Will Learn After This

- Build reusable form components
- Integrate frontend and backend validation gracefully
- Design consistent submission feedback

Common Mistakes to Avoid

- Duplicating form logic between create and edit
- Displaying generic errors only
- Losing form state on validation failure

Avoid these issues before requesting review.

Completion Criteria

- Create and edit flows are both functional
- Validation messages are clear and field-specific
- Submission behavior is deterministic

Review should be possible without extra clarifications.

Self-Check Questions

1. Where is shared form logic stored
2. How are backend validation errors displayed
3. How do you prevent duplicate submit

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 13: Frontend Testing (Jest and RTL)

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track Guided Assignment Page

Topic Overview

This assignment introduces frontend test coverage for render states, user behavior, and failure conditions.

Why This Matters in Real Projects

Frontend regressions are costly and visible. Deterministic UI tests protect user-critical workflows.

Visual Learning Map

Step 1
Plan test matrix for critical pages

Step 2
Mock backend responses

Step 3
Test success and failure interactions

Step 4
Run tests and document evidence

Prerequisite Reading (45-60 min)

- Jest Docs: Getting Started
- React Testing Library Intro
- MSW Docs

Assignment Tasks

1. Write tests for list page loading success empty and error
2. Write tests for form validation and submit flow
3. Mock API responses using MSW or axios mock adapter

Expected Deliverables

- Frontend test files with clear naming
- Test run output evidence
- docs/assignment-13.md

What You Will Learn After This

- Test user-visible behavior not implementation details
- Mock APIs in a controlled way
- Prevent flaky frontend tests

Common Mistakes to Avoid

- Using real network calls in unit tests
- Asserting internal state instead of UI output
- Not testing error states

Avoid these issues before requesting review.

Completion Criteria

- Tests are stable and pass consistently
- Critical user paths have both success and failure coverage
- Mock setup is documented

Review should be possible without extra clarifications.

Self-Check Questions

1. What behaviors are highest priority to test
2. How is API mocking configured
3. How do you detect flaky tests

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 14: End-to-End Feature Delivery (Rails and React)

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track Guided Assignment Page

Topic Overview

This assignment delivers one complete feature across backend and frontend: appointment reschedule with conflict protection.

Why This Matters in Real Projects

End-to-end ownership is key for role growth. It builds planning discipline, integration thinking, and quality accountability.

Visual Learning Map

Step 1
Define backend API change

Step 2
Implement frontend interaction flow

Step 3
Add tests across both layers

Step 4
Document end-to-end behavior

Prerequisite Reading (40-60 min)

- Rails Guides: Routing
- Rails Guides: Testing
- React Docs: Updating Arrays in State

Assignment Tasks

1. Add backend reschedule endpoint with overlap conflict handling
2. Add UI action and confirmation modal for reschedule
3. Add backend and frontend tests for success and conflict flows

Expected Deliverables

- Single PR containing backend and frontend changes
- Automated tests for new feature
- docs/assignment-14.md with demo steps

What You Will Learn After This

- Deliver integrated features confidently
- Coordinate contracts between backend and frontend
- Plan test strategy across layers

Common Mistakes to Avoid

- Changing API contract without frontend update
- Skipping conflict scenario tests
- Submitting feature without clear demo path

Avoid these issues before requesting review.

Completion Criteria

- Feature works end-to-end in local environment
- Conflict scenario is handled gracefully
- Evidence includes test and manual run

Review should be possible without extra clarifications.

Self-Check Questions

1. How do backend and frontend contracts align
2. What happens on schedule conflict
3. Which tests guard the new behavior

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 15: Database and API Performance Optimization

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track | Guided Assignment Page

Topic Overview

This assignment applies measurement-based optimization to improve one backend endpoint performance path.

Why This Matters in Real Projects

Performance work without measurement is guesswork. Teams need evidence-driven improvements with clear before and after metrics.

Visual Learning Map

Step 1
Capture baseline measurements

Step 2
Identify bottleneck from query profile

Step 3
Apply optimization and remeasure

Step 4
Validate no regression

Prerequisite Reading (45-60 min)

- Rails Guides: Active Record Query Interface
- PostgreSQL Docs: Using EXPLAIN
- Rails Guides: Caching with Rails

Assignment Tasks

1. Measure baseline latency for appointment listing
2. Fix one bottleneck such as N plus 1 query or missing index
3. Measure same endpoint after optimization

Expected Deliverables

- docs/assignment-15.md with before after table
- Query plan output screenshot
- Code changes implementing optimization

What You Will Learn After This

- Profile performance bottlenecks accurately
- Apply targeted optimizations with low risk
- Communicate improvement with evidence

Common Mistakes to Avoid

- Optimizing without baseline metrics
- Changing too many variables at once
- Ignoring correctness after optimization

Avoid these issues before requesting review.

Completion Criteria

- Improvement target reached for selected path
- No test regressions introduced
- Measurement method is documented clearly

Review should be possible without extra clarifications.

Self-Check Questions

1. How was bottleneck identified
2. Why was this optimization chosen
3. How did you ensure correctness remains intact

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 16: CI Pipeline and Quality Gates

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track Guided Assignment Page

Topic Overview

This assignment establishes automated quality checks for tests and linting in a repeatable CI workflow.

Why This Matters in Real Projects

CI gates protect main branch quality and provide fast feedback to engineers before merge.

Visual Learning Map

Step 1
Design CI job matrix

Step 2
Add backend and frontend test jobs

Step 3
Add lint checks for both stacks

Step 4
Validate failure behavior

Prerequisite Reading (35-50 min)

- GitHub Docs: Actions
- GitHub Docs: Workflow Syntax
- Docker Docs: Compose

Assignment Tasks

1. Add CI workflow with backend and frontend test jobs
2. Add lint checks for Ruby and JavaScript
3. Ensure CI fails on quality gate failure

Expected Deliverables

- .github/workflows/ci.yml
- CI run evidence screenshot or link
- docs/assignment-16.md

What You Will Learn After This

- Build practical CI pipelines
- Enforce standards automatically
- Diagnose CI failures quickly

Common Mistakes to Avoid

- Running tests only locally and not in CI
- Ignoring lint errors in pipeline
- Missing setup steps in CI environment

Avoid these issues before requesting review.

Completion Criteria

- Pipeline runs from clean checkout
- Failing checks block success status
- Workflow steps are documented

Review should be possible without extra clarifications.

Self-Check Questions

1. What are pipeline quality gates
2. How is CI environment bootstrapped
3. How does failing lint affect merge readiness

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 17: Production Readiness and Operational Runbook

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track Guided Assignment Page

Topic Overview

This assignment documents deployment safeguards and incident handling steps for common operational failures.

Why This Matters in Real Projects

Operational maturity is essential for safe releases. Teams need predictable runbooks for faster incident resolution.

Visual Learning Map

Step 1
Create deployment checklist

Step 2
Write incident runbook for API outage and queue backlog

Step 3
Create smoke-test script for post-deploy validation

Prerequisite Reading (40-55 min)

- Rails Guides: Configuring Rails
- Rails Guides: Securing Rails Applications
- The Twelve-Factor App

Assignment Tasks

1. Document migration and rollback checklist
2. Write runbook for API outage and Sidekiq backlog
3. Add scripts/smoke-test.sh for basic verification

Expected Deliverables

- docs/assignment-17.md operations guide
- scripts/smoke-test.sh
- Checklist table for release

What You Will Learn After This

- Prepare safe deploy procedures
- Respond systematically to common incidents
- Validate production readiness with simple checks

Common Mistakes to Avoid

- Runbooks with vague non-actionable steps
- No rollback strategy in checklist
- Smoke test that does not test critical paths

Avoid these issues before requesting review.

Completion Criteria

- Another engineer can run runbook independently
- Checklist includes rollback and verification
- Smoke test script executes successfully

Review should be possible without extra clarifications.

Self-Check Questions

1. What are first steps during API outage
2. How is rollback triggered safely
3. Which checks confirm system is healthy

← Previous Assignment | Table of Contents | Next Assignment →

Assignment 18: Capstone Demonstration and Technical Defense

Part of the ILA Rails and React Engineering Certification - Level 1 curriculum.

ILA Rails and React Track Guided Assignment Page

Topic Overview

This assignment synthesizes all prior work into a complete demonstration with architecture rationale and evidence-based defense.

Why This Matters in Real Projects

Engineering growth depends on clear communication of technical decisions, tradeoffs, and measurable outcomes.

Visual Learning Map

Step 1
Prepare end-to-end demo flow

Step 2
Collect evidence from tests logs and CI

Step 3
Explain design decisions and known limitations

Step 4
Answer reviewer questions

Prerequisite Reading (35-50 min)

- Google SRE Book
- AWS Well-Architected Framework
- Architecture Decision Records Guide

Assignment Tasks

1. Demonstrate full lifecycle create reschedule cancel
2. Present test coverage and CI status
3. Present logs and job execution evidence
4. Document three decisions and two limitations

Expected Deliverables

- docs/assignment-18.md final report
- Demo script or slides
- Final scorecard mapped to rubric

What You Will Learn After This

- Present technical work with confidence
- Defend decisions using data and code evidence
- Summarize limitations and next steps clearly

Common Mistakes to Avoid

- Making claims without evidence
- Overexplaining implementation and skipping outcomes
- Ignoring known limitations

Avoid these issues before requesting review.

Completion Criteria

- Every claim in demo has supporting evidence
- Tradeoffs are justified with context
- Reviewer questions are answered clearly

Review should be possible without extra clarifications.

Self-Check Questions

1. What was the most impactful design decision
2. Which risk remains unresolved and why
3. What would you improve next if given one week

← Previous Assignment | Table of Contents | Back to Table of Contents →